Лабораторная №1
Что показать и объяснить на защите

Сажин Федор Андреевич · М3216 · Подготовка 17.10.2026

1. Как сдавать по уточнениям преподавателя

Заранее отправить ссылку на репозиторий и тег lab1. Копия с историей создана в ai-in-sd-y29 под именем telegram-ai-lab1-sazhin-m3216. Начать разрешалось в личном аккаунте. Лабораторные №1, №2 и №3 сдаются отдельными контрольными точками.

На защите разрешены подготовленные файлы: можно открыть эту памятку, отчёт, исходники и результаты тестов. Надо понимать назначение кода и объяснять результат своими словами. Для первой лабы преподаватель отметил желательность запуска у себя со своими токенами.

Подлинная история запросов

Нужна настоящая переписка с исходными сообщениями студента. Сохранить её можно с помощью агента. Обязательного требования использовать именно OpenCode в присланном чате нет. Если проект разрабатывался с Codex, предъявляется фактическая история этой работы и указывается его участие.

В полученных архивах нет первоначальной переписки разработки. В комплекте сохранён фрагмент текущей проверки с дословными запросами пользователя и явно помеченным журналом действий. Он подтверждает эту проверку; первоначальную разработку за 13 октября им не подтверждают.

Что открыть перед защитой

README, submission/lab1-report.pdf, submission/checks.json, исходные ответы submission/experiment.json и настоящую переписку разработки. Не открывать .env с ключами при демонстрации экрана. На один токен должен работать один polling-процесс.

2. Удобный порядок живой демонстрации

Шаг	Действие	Что должно быть видно
1. Возможности	/start, затем /study и вопрос о списке и кортеже.	Справка без LLM; обычный вопрос обрабатывает учебный режим.
2. Контекст	«Приведи ещё один пример» без повторения темы.	Ответ учитывает историю текущего чата.
3. Режимы	/translate, «На английский: Сегодня я сдаю лабораторную работу». Затем /quiz, «Циклы Python» и ответ на вопрос.	Перевод и проверка знаний решают разные задачи. Смена режима очищает историю.
4. Два аккаунта	A: /settings 0.7 и вопрос о кортежах. B: /settings 1.0 и вопрос о циклах.	У каждого аккаунта собственные настройка и контекст.
5. Перезапуск	Остановить только бота и запустить снова из той же папки; с обоих аккаунтов показать /settings и продолжить разговор.	Режим, temperature и история восстановлены из той же БД.
6. Сброс	A: /reset, затем /settings. B: уточнить старый вопрос.	У A очищена только история, настройки сохранены; разговор B продолжается.
7. Длинный ответ	Заранее получить от модели ответ больше 4096 символов при достаточном LLM_MAX_TOKENS.	Несколько частей в правильном порядке. Если модель дала короткий ответ, сценарий ещё не подтверждён.
8. Отказ	На отдельной конфигурации проверить неверный ключ API; показать HTTP-тесты и восстановить рабочую настройку.	Понятная ошибка без ключа и traceback, процесс продолжает работать.

Не удалять volume и не переносить папку в середине проверки перезапуска. Тесты PostgreSQL используют отдельные временные контейнеры и не затрагивают пользовательскую БД.

Живой показ двух реальных Telegram-аккаунтов и длинного ответа остаётся за студентом. Автоматические проверки кода не являются записью показа в Telegram.

3. Ответы на вопросы об отказах

Ситуация	Фактическое поведение	Короткое объяснение
Модель не отвечает	Весь вызов ограничен LLM_TIMEOUT, по умолчанию 60 с, включая ожидание повторов.	Безопасная ошибка о таймауте; успешная пара не записывается.
HTTP 429, 5xx или ошибка сети	Не более LLM_RETRIES повторов; по умолчанию один. Затем сообщение об ошибке.	Повторы ограничены. В эксперименте они отключены: один запуск соответствует одному запросу.
Неверный ключ: 401/403	Сообщение об отклонённом ключе без повторов.	Повтор с тем же ключом не исправляет доступ. Ключ и тело ответа провайдера не выводятся.
HTTP 402	Сообщение об отклонении из-за баланса или лимита.	Код не обещает точную причину ограничений аккаунта без данных провайдера.
Пустой ответ, плохой JSON, обрыв	Проверяются choices, message.content и finish_reason. Незавершённый ответ отклоняется.	Он не сохраняется как успешный и не заменяется пустым сообщением Telegram.
PostgreSQL недоступна	При запуске процесс завершается; во время диалога выдаётся безопасная ошибка. Healthcheck показывает 503.	Нет скрытого перехода на память. После восстановления используется сохранённая БД.
Telegram недоступен	Начальные вызовы Telegram ограничены 30 с. Позже доставка может завершиться ошибкой.	Через недоступный Telegram нельзя сообщить об отказе самого Telegram. Пара уже может находиться в БД: запись предшествует доставке.
Слишком большой ввод	Проверяются INPUT_MAX_CHARS и бюджет контекста. Не помещающийся вопрос отклоняется до API.	Текущий вопрос не обрезается. Старые пары удаляются целиком; инструкция и few-shot сохраняются.

Места в коде: app/llm.py, app/handlers/assistant.py, app/history.py, app/health.py и app/storage.py. HTTP-тесты используют настоящий локальный HTTP-сервер с управляемыми ответами и не требуют настоящего ключа.

4. Как объяснить архитектуру и эксперимент

```
Текст из личного чата
  -> Router: команда или обычное сообщение
  -> Assistant: блокировка своего чата, профиль и история
  -> instructions + история по порядку + текущий вопрос
  -> HTTPModel: API, temperature, таймаут, проверка ответа
  -> транзакция PostgreSQL: полная успешная пара
  -> split_reply: части до 4096 единиц UTF-16
  -> Telegram
```

Почему данные пользователей не смешиваются?

Каждый запрос к профилю и истории ограничен chat_id. lab1_profiles хранит режим и temperature; lab1_turns - пары вопрос-ответ с возрастающим id. Внутри процесса сообщения одного чата обрабатываются последовательно. /reset и смена режима используют тот же механизм ожидания.

Зачем few-shot и что такое R.C.T.F.?

В /translate два примера показывают готовый перевод и уточнение отсутствующего языка. Role задаёт роль, Context - учебную ситуацию, Task - действие, Format - форму результата. Промпт помогает управлять ответом, но не является проверкой прав или хранилищем секретов.

Что показал эксперимент?

Для каждой temperature 0.0, 0.3, 0.7, 1.0 сохранены три независимых ответа на один вопрос. Перед каждым очищалась отдельная история через Assistant.reset. Модель, инструкция и max_tokens=8000 одинаковы. Сходство текстов снизилось с 30.0% до 15.0%; длина не росла монотонно. Нумерация соблюдена в 11 из 12 ответов.

Предварительно выбрана 0.3: краткие ответы и соблюдённая нумерация в этой выборке. Это не гарантия точности: при 0.3 модель ошиблась в правиле записи кортежа. Temperature=0 не дала одинаковых ответов. По трём повторам нельзя обещать устойчивый результат для всех задач.

Какой совет AI пришлось исправить?

В старом отчёте описана смена неработавшей бесплатной модели после ошибок. В текущей проверке исправлен ещё один недочёт исходного AI-комплекта: добавление LLM-параметров в .env.cloud не обеспечивало их передачу контейнеру. Параметры добавлены в compose.yaml и проверены через docker compose config.

Сохранение пары до доставки, приблизительная оценка контекста и возможность фактических ошибок модели - известные ограничения. Их нужно объяснить, а не обещать отсутствие сбоев.